COLLEGE PROJECT DOCUMENTATION • AIML TRACK

Student Performance Prediction System

An End-to-End Machine Learning Solution for Educational Analytics

DEGREE COURSE	DOMAIN	CORE ALGORITHM	REPOSITORY
B.Tech Information Technology	AI, Machine Learning & Full-Stack	Random Forest (Ensemble)	github.com/siddharthkhilari

Academic Disclaimer: This application is developed strictly for educational research, student analysis, and college project evaluation. It serves as an assistive mentoring indicator and should not be used as the sole determinant for formal university grading decisions.

1. Project Objective & Problem Statement

In conventional university workflows, student evaluation is purely retrospective: academic struggles are only discovered after semester examination results are released, when it is already too late for corrective mentoring.

The **Student Performance Prediction System** solves this problem by providing a predictive, proactive decision-support tool. It models a student's academic habits and behavioral metrics during the semester to forecast expected final examination marks (0–100%) and classify their achievement tier (*Excellent*, *Good*, *Average*, or *Needs Improvement*). Furthermore, the system synthesizes **natural-language analytical explanations** to help faculties understand the root causes behind a student's risk factors.

2. Languages, Libraries & Technologies Used

Category	Languages & Frameworks	Role in Project
Frontend	JavaScript (ES6+), React 18, Vite, HTML5, CSS3	Interactive Single Page Application (SPA), state management, dark/light theme switching, and live form validation.
Data Visualization	Recharts & Lucide React	Interactive graphical charts: Category Distribution Pie Chart, Study Hours vs. Marks Bar Chart, and Attendance vs. Marks Scatter Plot.
Backend API	Python 3.13, Flask REST API, Flask-CORS	Lightweight, high-performance RESTful API service handling request routing, payload validation, inference execution, and CSV file exports.
Machine Learning	Scikit-Learn, Pandas, NumPy, Joblib	Data preprocessing, multi-variable correlation generation, Random Forest model training, hyperparameter configuration, and binary model serialization.

Category	Languages & Frameworks	Role in Project
Database & ORM	SQLite & SQLAlchemy ORM	Zero-configuration local relational database for persisting student parameters, timestamps, prediction scores, and explanations without external cloud servers.
Automated Testing	Pytest 9.0	12 automated unit and integration tests covering dataset generation, input boundaries, CRUD operations, and REST endpoints.

3. Key Engineering Skills Demonstrated

Machine Learning & Data Science

- Supervised Ensemble Learning (Random Forest Regressor & Classifier).
- Zero Data Leakage architecture (target variables strictly excluded from training features).
- Rigorous statistical evaluation on isolated test sets (MAE, RMSE, R^2).
- Feature importance ranking and explainable AI heuristics.

Full-Stack Software Architecture

- Decoupled Client-Server RESTful communication.
- Relational database schema design and ORM transactions with SQLAlchemy.
- Responsive, mobile-friendly component design with CSS custom properties.
- Automated test-driven verification with 100% test coverage pass rate.

4. How the Project Works (System Architecture & Pipeline)

The complete end-to-end prediction pipeline operates across five synchronized stages:

1. **Data Input & Boundary Validation:** The faculty or student inputs 7 key parameters: *Previous Marks (0–100%)*, *Class Attendance (0–100%)*, *Daily Study Hours (0–16h)*, *Assignment Completion (0–100%)*, *Backlog Count (0–20)*, *Sleep Hours (0–24h)*, and *Extracurricular Participation (Yes/No)*. The inputs are validated strictly on both client-side and server-side to reject invalid, negative, or infinite values.
2. **REST API Transmission:** The validated payload is dispatched via an asynchronous HTTP POST request (`/api/predict`) from the React frontend to the Flask REST backend.
3. **Machine Learning Model Inference:** The backend deserializes the trained **Random Forest Regressor** (`marks_predictor.joblib`) from memory. The model generates a continuous numerical score representing expected marks, clamped to [0.0, 100.0].
4. **Academic Tier Classification:** The predicted score is mapped to one of four standardized college achievement tiers:
   • Excellent ($\ge 80\%$) • Good ($70\% - 79.9\%$) • Average ($50\% - 69.9\%$) • Needs Improvement ($< 50\%$).
5. **Analytical Explanation Synthesis:** A rule-based reasoning engine evaluates the student's specific feature weights and formulates an actionable summary highlighting positive drivers (e.g., strong attendance, optimal sleep) and risk flags (e.g., active backlogs, insufficient study blocks).
6. **Persistent Storage & UI Presentation:** The prediction record is saved to SQLite via SQLAlchemy with an ISO UTC timestamp. The frontend renders the result card, updates real-time KPI cards, and refreshes the interactive Recharts distribution graphs.

5. Machine Learning Dataset & Verified Evaluation Metrics

The model was trained on **1,500 realistic educational records** using a documented multi-factor formula with Gaussian variance and a reproducible random seed (`random_state=42`). 80% of data (1,200 records) was used for model training, and 20% (300 records) was held out strictly for independent testing.

Evaluation Metric	Measured Score	Technical Meaning & Educational Significance
R^2 Score (Coefficient of Determination)	**71.78% (0.718)**	Measures variance explained by the model on unseen students; indicates strong predictive reliability without overfitting.
Mean Absolute Error (MAE)	**3.525 Marks**	The average absolute difference between the predicted mark and the actual exam score is only ~3.5 marks out of 100.
Root Mean Squared Error (RMSE)	**4.485 Marks**	Penalizes larger outliers; confirms consistent low variance across all student score ranges.
Tier Classification Accuracy	**84.33%**	Percentage of test students classified into the exact correct achievement category.

Feature Importance Ranking (from Random Forest Estimators)

The trained ensemble confirmed the highest statistical weights belong to: **1. Previous Examination Marks (38.4%)**, **2. Class Attendance (24.1%)**, **3. Daily Study Hours (18.6%)**, **4. Active Backlogs (11.2%)**, and **5. Assignment Completion (5.1%)**.

6. Application Modules & REST API Endpoints

API Endpoint	Method	Description & Functionality
`/api/health`	GET	Verifies backend server health, database status, and model persistence.
`/api/dashboard`	GET	Returns aggregated KPIs, category breakdown, and scatter correlation data.
`/api/predict`	POST	Validates student parameters, executes ML inference, saves record, and returns results.
`/api/students`	GET	Retrieves saved records with optional search (`?search=`) and category filter (`?category=`).
`/api/students/<id>`	DELETE	Permanently removes a student record after user confirmation.
`/api/history`	GET	Fetches sequential chronological audit log of all generated predictions.
`/api/model-metrics`	GET	Supplies real MAE, RMSE, R^2, and feature importance percentages to the UI.
`/api/export`	GET	Streams a downloadable CSV spreadsheet attachment containing all saved student records.

7. Project Execution & Demonstration Instructions

The project runs locally in Windows PowerShell and VS Code without paid cloud services or setup fees:

Terminal 1: Start Flask Backend Server

```
cd "C:\Users\Admin\OneDrive\Desktop\student-performance-prediction"
.\backend\venv\Scripts\Activate.ps1
python backend\app.py
```

Backend starts listening at: `http://127.0.0.1:5000`

Terminal 2: Start React Vite Frontend

```
cd "C:\Users\Admin\OneDrive\Desktop\student-performance-prediction\frontend"
npm run dev
```

Frontend web app opens at: `http://localhost:5173`

8. Summary for College Teacher / Examiner

Key Answers to Common Viva / Evaluation Questions:

• **Why Random Forest over Linear Regression?** Random Forest naturally captures non-linear relationships (like the diminishing returns of studying 12+ hours or non-linear penalties for multiple backlogs) and uses bagging to resist overfitting.

• **How is data leakage avoided?** Final marks and roll numbers are strictly removed from the input matrix X before model fitting, ensuring the model never cheats during evaluation.

• **What is the real-world application?** Academic advising cells and college mentors can upload student data mid-semester to identify at-risk students 8–10 weeks prior to exams, scheduling remedial coaching in time.

Student Performance Prediction System • B.Tech IT GitHub: https://github.com/siddharthkhilari/student-performance-prediction